



Lógica em Ação •

Estruturas de Repetição

Aprenda a automatizar listas, calcular carrinhos e filtrar produtos com Loops (`for`, `while`, `.map`, `.reduce`)



Loops & Iteração



Acumulador de Totais



Filtros de Produtos



4 Desafios Práticos



OneCompiler Ready



12 Questões com Gabarito



1. Por que precisamos de Repetição na Programação?



O Problema da Duplicação Manual:

Imagine que sua lanchonete tem **100 produtos** cadastrados. Sem estruturas de repetição, você teria que escrever 100 linhas de código repetidas para calcular o total ou desenhar os cards. Com um **loop**, você escreve o cálculo **uma única vez** e o computador repete para toda a lista em milissegundos!



Loop `for` Clássico

Ideal quando sabemos o tamanho da lista: `for (let i = 0; i < itens.length; i++)`. Percorre item por item do carrinho.



Loop `while`

Repete *enquanto* uma condição for verdadeira: `while (estoque > 0)`. Muito usado em simulações e filas de processamento.

Métodos `.map()` e `.filter()`

Funções modernas do JavaScript que transformam listas em HTML na tela e filtram lanches por preço automaticamente.

2. Laboratórios de Código com Desafios Práticos

Abra os exemplos no OneCompiler, execute o código original e depois **realize a missão de alteração** proposta em cada laboratório:

Lab 1: O Somador do Carrinho (Loop `for` com Variável Acumuladora)

Conceito: A variável `let total = 0` acumula a soma de cada item: `total = total + (item.preco * item.qtd)` a cada volta do loop.

MISSÃO DO ALUNO (Alteração no Código):

1. Clique em " **Abrir no OneCompiler**".
2. No array carrinho, adicione um 4º item: `{ nome: "Batata Frita", preco: 14.00, qtd: 1 }`.
3. Adicione uma **taxa de entrega fixa de R\$ 8,00** somada ao total final e execute novamente!

 `lab1-somador-carrinho.html`

 Copiar

 Abrir no OneCompiler

```
<!DOCTYPE html>
<html lang="pt-BR">
<head>
```

```

<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.
<title>Lab 1 - Somador do Carrinho (Loop for)</title>
<style>
  body { font-family: sans-serif; background: #f8fafc; padding: 2
  .box { background: white; border-radius: 14px; padding: 20px; m
  ul { list-style: none; padding: 0; margin: 10px 0; font-size: 0
  li { padding: 6px 0; border-bottom: 1px solid #eee; display: fl
  button { background: #2563eb; color: white; border: none; paddi
  .total-box { font-size: 1.2rem; font-weight: bold; color: #16a3
</style>
</head>
<body>
  <div class="box">

```

🔍 Lab 2: Filtro de Preço Máximo (Loop `for` combinado com Decisão `if`)

Conceito: O loop percorre 5 produtos do cardápio e a condição `if (produto.preco <= limite)` seleciona apenas os lanches econômicos.

🎯 MISSÃO DO ALUNO (Alteração no Código):

1. Abra no OneCompiler.
2. Mude a variável `limiteMaximo` de `30.00` para **`20.00`**.
3. No CSS dentro de `<style>`, mude a cor de fundo dos itens em promoção de `#dcfce7` (verde) para **`#fef3c7` (amarelo)**.

📄 lab2-filtro-loop.html

📋 Copiar

▶ Abrir no OneCompiler

```

<!DOCTYPE html>
<html lang="pt-BR">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.
  <title>Lab 2 - Filtro de Preço Máximo</title>
  <style>

```

```

        body { font-family: sans-serif; background: #fdfaf6; padding: 20px; }
        .card { background: white; border: 1px solid #cbd5e1; border-radius: 10px; padding: 10px; }
        .item-promo { background: #dcfce7; padding: 8px 12px; border-radius: 5px; }
    </style>
</head>
<body>
    <div class="card">
        <h3>🔍 Lanches Econômicos (Até R$ 30)</h3>
        <p style="font-size:0.8rem; color:#64748b;">Filtro dinâmico com
        <div id="resultado-filtro"></div>
    </div>
</body>

```

📱 Lab 3: Gerador Automático de Cardápio com HTML (.map)

Conceito: A função `.map()` percorre cada objeto da lista e o transforma em uma estrutura HTML `<div class="card">` pronta para ser desenhada no DOM.

🎯 MISSÃO DO ALUNO (Alteração no Código):

1. Abra no OneCompiler.
2. Adicione um novo produto ao array: `{ id: 4, nome: "Sorvete de Chocolate", preco: 12.00, img: "🍦" }`.
3. Adicione um botão `<button>Comprar</button>` dentro do card gerado pelo JavaScript.

📄 lab3-cardapio-dinamico.html

📋 Copiar

▶ Abrir no OneCompiler

```

<!DOCTYPE html>
<html lang="pt-BR">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Lab 3 - Cardápio Dinâmico (.map)</title>
    <style>
        body { font-family: sans-serif; background: #f8fafc; padding: 20px; }
        .grid { display: grid; grid-template-columns: repeat(auto-fit, minmax(300px, 1fr)); gap: 10px; }
        .card-prod { background: white; border: 1px solid #e2e8f0; border-radius: 10px; padding: 10px; }
    </style>
</head>
<body>
    <div class="grid">
        <div class="card-prod">
            <img alt="🍦" />
            <h3>Sorvete de Chocolate</h3>
            <p>R$ 12.00</p>
            <button>Comprar</button>
        </div>
    </div>
</body>

```

```

        .emoji { font-size: 2.5rem; }
        .preco { color: #16a34a; font-weight: bold; margin-top: 4px; }
    </style>
</head>
<body>
    <h3 style="text-align:center; color:#1e1b4b;">🍔 Cardápio Gerado Au
    <div class="grid" id="grid-cardapio"></div>

```

⌚ Lab 4: Simulador de Fila de Preparo na Cozinha (Loop `while`)

Conceito: O loop `while (pedidosPendentes > 0)` simula a cozinha preparando pedidos um a um até que a fila seja zerada.

🎯 MISSÃO DO ALUNO (Alteração no Código):

1. Abra no OneCompiler.
2. Altere o número inicial de pedidos de 5 para 8.
3. Adicione uma mensagem no final informando: *"Cozinha fechada para higienização!"*.

📄 lab4-fila-cozinha-while.html

📋 Copiar

▶ Abrir no OneCompiler

```

</head>
<body>
    <div class="painel">
        <h2>👨‍🍳 Fila de Produção da Cozinha</h2>

        <div id="status-cozinha" style="margin: 14px 0;"></div>
    </div>

    <script>
        let pedidosNaFila = 5;
        let logHTML = "";

        // LOOP WHILE: Continua executando ENQUANTO houver pedidos pend
        while (pedidosNaFila > 0) {
            logHTML += '<div class="log-linha">🔍 Preparando pedido #'
            pedidosNaFila--; // Decrementa a fila até atingir zero

```



3. Exercício de Fixação: Estruturas de Repetição (12 Questões)

Responda às 12 questões objetivas sobre loops e manipulação de listas. Clique em "**Verificar Respostas**" ao final:

1 Por que um aplicativo de pedidos utiliza estruturas de repetição (loops) em vez de escrever tags manuais para cada produto?

- ☒ Porque o cardápio possui tamanho dinâmico e o loop percorre a lista automaticamente sem duplicar código
- ☐ Porque o JavaScript é proibido de exibir menos de 10 produtos na tela
- ☐ Para diminuir a resolução das fotos dos lanches
- ☐ Para desligar o banco de dados do restaurante

2 Quais são as três partes obrigatórias que compõem o cabeçalho de um loop clássico ``for`` (inicialização; condição; incremento) `?`

A variável contadora inicial (ex: ``let i = 0``), a condição de parada (ex: ``i < lista.length``) e o passo de avanço (ex: ``i++``)

- ☐ O nome do cliente, o CPF e a senha do banco
- ☐ O título HTML, o estilo CSS e a imagem do produto
- ☐ Apenas a cor da fonte do texto

3 No Lab 1, qual é a função da variável acumuladora ``let total = 0;`` declarada antes do loop ``for``?

Iniciar zerada e acumular a soma do subtotal de cada produto a cada volta do loop

- ☐ Apagar os itens do carrinho de compras
- ☐ Cancelar o pedido do cliente
- ☐ Desconectar o celular da rede Wi-Fi

4 O que acontece em um programa se a condição de um loop ``while`` nunca se tornar falsa (por exemplo, esquecer de decrementar o contador)?

- ☒ Ocorre um "loop infinito", travando a execução e congelando a aba do navegador

- ☐ O computador calcula o dobro da velocidade
- ☐ O código é corrigido automaticamente pelo CSS
- ☐ O pedido é entregue instantaneamente na casa do cliente

5 Se um array de produtos possui 4 itens cadastrados, qual é o valor da propriedade ``carrinho.length`` e qual é o índice do último item?

- ☒ O tamanho (length) é 4 e o índice do último item é 3 (pois a contagem inicia em 0)

- ☐ O tamanho é 4 e o último índice é 4
- ☐ O tamanho é 10 e o último índice é 1
- ☐ O tamanho é sempre zero

6 Qual é a finalidade da instrução ``i++`` colocada no final do loop ``for``?

- ☒ Incrementar (somar +1) ao contador ``i`` para que o loop avance para o próximo item da lista

- ☐ Multiplicar o preço por dois
- ☐ Diminuir o estoque para zero
- ☐ Fechar a janela do formulário

7 No Lab 2, qual estrutura foi combinada dentro do loop ``for`` para filtrar e exibir apenas lanches abaixo de R\$ 30,00?

- ☒ A condicional ``if (item.preco <= limiteMaximo)``

- ☐ A tag de imagem ``<img``
- ☐ A propriedade ``border-radius``
- ☐ O método ``window.print()``

8 No Lab 3, o que a função ``produtos.map(p => ...)`` realiza com o array de dados do cardápio?

- ☒ Transforma cada objeto do cardápio em um bloco de código HTML formatado com foto, título e preço

- ☐ Exclui todos os lanches do cardápio
- ☐ Desliga o monitor do computador
- ☐ Converte números decimais em texto aleatório

- 9 O comando ``break`` pode ser utilizado dentro de um loop de busca para interromper imediatamente a repetição assim que o produto procurado for encontrado, economizando processamento.

☒ Verdadeiro

☐ Falso

- 10 Em JavaScript, os índices de um array começam no número 1, logo o primeiro produto de uma lista de compras é acessado através de ``itens[1]``.

☐ Verdadeiro

☒ Falso

- 11 Se um loop for escrito como ``for (let i = 0; i < 5; i++)``, quantas vezes o bloco de código interno será executado?

☒ Exatamente 5 vezes (com i valendo 0, 1, 2, 3 e 4)

☐ Apenas 1 vez

☐ 50 vezes

☐ Nenhuma vez

- 12 A combinação de loops com manipulação de DOM (``document.getElementById().innerHTML += ...``) permite criar interfaces dinâmicas que se expandem automaticamente conforme novos itens são adicionados à lista.

☒ Verdadeiro

☐ Falso

☒ Verificar Respostas

☐ Limpar Respostas

Relatório de Desempenho do Aluno

12 / 12

☒ Questão 1: Correta!

Estruturas de repetição automatizam o processamento de listas com tamanho dinâmico sem duplicar código.

☒ Questão 2: Correta!

O for clássico exige: inicialização do contador (let $i = 0$), condição de parada ($i < lista.length$) e o incremento ($i++$).

☒ Questão 3: Correta!

A variável acumuladora começa em zero e soma o subtotal de cada produto a cada volta do loop.

☒ Questão 4: Correta!

Sem uma condição que se torne falsa, o loop roda infinitamente e consome toda a memória da aba do navegador.

☒ Questão 5: Correta!

O length indica a quantidade total de itens (4), mas os índices do array vão de 0 até 3 (4 - 1).

✓ **Questão 6:** Correta!

A instrução `i++` adiciona +1 ao índice a cada repetição para que o loop avance até o próximo item.

✓ **Questão 7:** Correta!

Combinar o loop com a condicional `if` permite filtrar dados que atendem a um critério específico.

✓ **Questão 8:** Correta!

O método `.map()` itera sobre cada elemento do array e retorna uma nova lista de strings HTML formatadas.

✓ **Questão 9:** Correta!

Verdadeiro. O comando `break` encerra o loop no momento exato em que o item desejado é encontrado.

✓ **Questão 10:** Correta!

Falso. Em JavaScript e na maioria das linguagens, os índices de um array sempre iniciam na posição 0.

✓ **Questão 11:** Correta!

O loop inicia em `i = 0` e repete enquanto `i < 5`, executando para `i = 0, 1, 2, 3` e `4` (5 vezes no total).

✓ **Questão 12:** Correta!

Verdadeiro. Loops combinados com `innerHTML` criam interfaces ricas que renderizam catálogos inteiros dinamicamente.



Imprimir / Salvar em PDF



ATENÇÃO ALUNOS: Após testar os 4 códigos no OneCompiler, realizar as missões de alteração e concluir o Quiz, envie seus arquivos HTML modificados pelo formulário oficial de entrega.



Formulário Oficial de Entrega



Formulário de Entrega do Relatório

Envio dos exercícios modificados + Desafios de Loops



Material Pedagógico: Estruturas de Repetição • Disciplina: Programação Mobile • 2026



Topo